

**LAPORAN HASIL PENGUJIAN LOAD BALANCING
ANALISIS PERBANDINGAN SKENARIO i4 DAN i7 MENGGUNAKAN
ALGORITMA ROUND ROBIN DAN LEAST CONNECTIONS**

Dosen Pengampu : RUNAL REZKIAWAN, S.Kom., M.T

Nama : Muh. Yahya Al-Qadri

NIM : 105841120523

Kelas : 6-B

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH MAKASSAR**

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Allah SWT, karena atas rahmat, hidayah, dan karunia-Nya, penulis dapat menyelesaikan laporan hasil pengujian load balancing ini dengan baik.

Laporan ini disusun sebagai bentuk dokumentasi praktikum pada mata kuliah Scalable Systems Design. Pengujian dilakukan dengan membangun simulasi load balancing menggunakan Docker, Nginx, dan Python Flask. Sistem diuji pada dua skenario jumlah backend, yaitu i4 dengan empat server backend dan i7 dengan tujuh server backend. Setiap skenario diuji menggunakan dua algoritma load balancing, yaitu Round Robin dan Least Connections.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan untuk perbaikan pada laporan dan pengembangan sistem selanjutnya. Semoga laporan ini dapat memberikan manfaat dan menjadi referensi dalam memahami konsep load balancing pada sistem terdistribusi.

Makassar, 20 Juni 2026

NurSyahlan Ruslan

DAFTAR ISI

Contents

KATA PENGANTAR.....	2
BAB I.....	4
PENDAHULUAN	4
A. Latar Belakang.....	4
B. Rumusan Masalah.....	4
C. Tujuan Praktikum	4
BAB II	5
LANDASAN TEORI	5
A. Load Balancing.....	5
B. Algoritma Round Robin	5
C. Algoritma Least Connections	5
D. Docker, Nginx, dan Flask	5
BAB III	6
METODOLOGI DAN PERANCANGAN SISTEM	6
A. Spesifikasi Lingkungan Pengujian	6
B. Arsitektur Sistem	6
C. Implementasi dan Source Code	6
D. Metode Pengujian	12
BAB IV	14
HASIL DAN PEMBAHASAN	14
A. Penempatan Gambar Hasil Pengujian	14
B. Hasil Pengujian i4.....	14
BAB V	22
KESIMPULAN	22

BAB I

PENDAHULUAN

A. Latar Belakang

Perkembangan aplikasi berbasis web menyebabkan kebutuhan sistem yang mampu menangani banyak request secara bersamaan semakin meningkat. Apabila seluruh request hanya diarahkan ke satu server, maka server tersebut berpotensi mengalami kelebihan beban, waktu respons meningkat, bahkan dapat menyebabkan layanan tidak dapat diakses.

Salah satu solusi yang digunakan untuk meningkatkan skalabilitas sistem adalah load balancing. Load balancing bekerja dengan mendistribusikan request dari client ke beberapa server backend. Dengan cara ini, beban kerja tidak hanya ditanggung oleh satu server, tetapi dibagi ke beberapa server yang tersedia.

Pada praktikum ini dilakukan simulasi load balancing menggunakan Docker, Nginx, dan Flask. Pengujian dirancang dalam dua variasi jumlah server backend, yaitu i4 dan i7. i4 menunjukkan skenario dengan empat server backend, sedangkan i7 menunjukkan skenario dengan tujuh server backend. Kedua skenario tersebut diuji menggunakan algoritma Round Robin dan Least Connections untuk melihat perbedaan pola distribusi request.

B. Rumusan Masalah

1. Bagaimana mengimplementasikan simulasi load balancing menggunakan Docker, Nginx, dan Python Flask?
2. Bagaimana pola distribusi request pada skenario i4 dan i7 menggunakan algoritma Round Robin?
3. Bagaimana pola distribusi request pada skenario i4 dan i7 menggunakan algoritma Least Connections?
4. Bagaimana perbandingan karakteristik distribusi beban antara Round Robin dan Least Connections?

C. Tujuan Praktikum

1. Membangun simulasi load balancing berbasis Docker, Nginx, dan Flask.
2. Menguji distribusi request pada skenario i4 dan i7.
3. Menganalisis perbedaan algoritma Round Robin dan Least Connections.
4. Menyusun dokumentasi hasil pengujian sebagai bahan laporan praktikum.

BAB II

LANDASAN TEORI

A. Load Balancing

Load balancing merupakan teknik pembagian beban request dari client ke beberapa server backend. Load balancer bertindak sebagai pintu masuk utama yang menerima request, kemudian menentukan server backend yang akan memproses request tersebut. Tujuan utama load balancing adalah meningkatkan ketersediaan layanan, mengurangi risiko bottleneck, serta menjaga kestabilan sistem ketika menerima banyak request.

B. Algoritma Round Robin

Round Robin merupakan algoritma load balancing yang membagi request secara bergiliran berdasarkan urutan daftar server backend. Jika terdapat empat server backend, maka request akan diarahkan secara berputar dari server pertama sampai server keempat, kemudian kembali lagi ke server pertama.

Kelebihan Round Robin adalah sederhana dan mudah diterapkan. Namun, algoritma ini tidak memperhatikan jumlah koneksi aktif atau kondisi server pada saat request dikirim. Oleh karena itu, Round Robin lebih cocok digunakan pada lingkungan server yang memiliki kemampuan dan beban kerja relatif sama.

C. Algoritma Least Connections

Least Connections merupakan algoritma load balancing yang memilih server backend berdasarkan jumlah koneksi aktif paling sedikit. Request baru akan diarahkan ke server yang sedang memiliki antrean atau koneksi aktif paling rendah. Algoritma ini lebih dinamis dibandingkan Round Robin karena memperhatikan kondisi server pada saat request diterima oleh load balancer.

Least Connections cocok digunakan pada sistem yang memiliki request dengan durasi pemrosesan berbeda-beda. Pada kondisi tersebut, server yang lebih sibuk tidak terus-menerus menerima request baru, sehingga potensi penumpukan beban dapat dikurangi.

D. Docker, Nginx, dan Flask

Docker digunakan untuk menjalankan aplikasi di dalam kontainer yang terisolasi. Pada simulasi ini, Docker Compose digunakan untuk menjalankan beberapa kontainer sekaligus, yaitu satu kontainer Nginx sebagai load balancer dan beberapa kontainer Flask sebagai backend server.

Nginx digunakan sebagai load balancer yang menerima request dari client melalui port 8085. Flask digunakan sebagai aplikasi backend sederhana yang menampilkan identitas server, hostname kontainer, dan response delay. Informasi tersebut digunakan untuk membuktikan server mana yang sedang memproses request.

BAB III

METODOLOGI DAN PERANCANGAN SISTEM

A. Spesifikasi Lingkungan Pengujian

Sistem operasi/lingkungan kerja: GitHub Codespaces atau terminal berbasis Linux.

Platform kontainer: Docker dan Docker Compose.

Load balancer: Nginx Alpine.

Backend server: Python 3.9-slim dan Flask.

Port akses client: 8085.

Jumlah server backend: i4 sebanyak 4 server dan i7 sebanyak 7 server.

Metode pengujian: request paralel menggunakan Python ThreadPoolExecutor.

B. Arsitektur Sistem

Arsitektur sistem terdiri dari satu load balancer dan beberapa server backend. Client mengakses layanan melalui alamat localhost:8085 atau URL port forwarding Codespaces. Request dari client diterima oleh Nginx, kemudian diteruskan ke salah satu server backend sesuai algoritma yang aktif.

Pada skenario i4, sistem menggunakan empat backend, yaitu i4-web-1, i4-web-2, i4-web-3, dan i4-web-4. Pada skenario i7, sistem menggunakan tujuh backend, yaitu i7-web-1 sampai i7-web-7. Setiap backend menjalankan aplikasi Flask yang sama, tetapi memiliki identitas server dan delay yang berbeda.

C. Implementasi dan Source Code

Bagian ini digunakan untuk menempatkan screenshot source code. Screenshot source code sebaiknya diambil dari VS Code agar terlihat bahwa program benar-benar dibuat dan dijalankan. Apabila dosen meminta bukti kode program, bagian ini menjadi tempat paling tepat untuk meletakkan screenshot tersebut.

```

1 services:
2   nginx-lb:
3     image: nginx:alpine
4     container_name: i4-roundrobin-nginx-lb
5     ports:
6       - "8085:80"
7     volumes:
8       - ./nginx/i4-roundrobin.conf:/etc/nginx/nginx.conf:ro
9     depends_on:
10      - i4-web-1
11      - i4-web-2
12      - i4-web-3
13      - i4-web-4
14     networks:
15       - i4-roundrobin-network
16
17 i4-web-1:
18   image: python:3.9-slim
19   container_name: i4-roundrobin-web-1
20   environment:
21     - SERVER_NAME=WEB SERVER 1 (i4-web-1)
22     - SERVER_GROUP=I4
23     - SERVER_COLOR=#00d4ff
24     - PROCESS_DELAY=0.30
25   volumes:
26     - ./app:/app
27   working_dir: /app
28   command: sh -c "pip install --no-cache-dir flask && python app.py"
29   networks:
30     - i4-roundrobin-network
31
32 i4-web-2:
33   image: python:3.9-slim
34   container_name: i4-roundrobin-web-2
35   environment:
36     - SERVER_NAME=WEB SERVER 2 (i4-web-2)
37     - SERVER_GROUP=I4
38     - SERVER_COLOR=#00ff99
39     - PROCESS_DELAY=0.10
40   volumes:
41     - ./app:/app
42   working_dir: /app
43   command: sh -c "pip install --no-cache-dir flask && python app.py"
44   networks:
45     - i4-roundrobin-network
46
47 i4-web-3:
48   image: python:3.9-slim
49   container_name: i4-roundrobin-web-3
50   environment:
51     - SERVER_NAME=WEB SERVER 3 (i4-web-3)
52     - SERVER_GROUP=I4
53     - SERVER_COLOR=#ffd166
54     - PROCESS_DELAY=0.20
55   volumes:
56     - ./app:/app
57   working_dir: /app
58   command: sh -c "pip install --no-cache-dir flask && python app.py"
59   # ... konfigurasi web-3 dan web-4 memakai pola yang sama ...

```

Gambar 3.1 Source Code docker-compose untuk Skenario i4

```

1 services:
2   nginx-lb:
3     image: nginx:alpine
4     container_name: i7-leastconn-nginx-lb
5     ports:
6       - "8085:80"
7     volumes:
8       - ./nginx/i7-leastconn.conf:/etc/nginx/nginx.conf:ro
9     depends_on:
10      - i7-web-1
11      - i7-web-2
12      - i7-web-3
13      - i7-web-4
14      - i7-web-5
15      - i7-web-6
16      - i7-web-7
17     networks:
18       - i7-leastconn-network
19
20 i7-web-1:
21   image: python:3.9-slim
22   container_name: i7-leastconn-web-1
23   environment:
24     - SERVER_NAME=WEB SERVER 1 (i7-web-1)
25     - SERVER_GROUP=I7
26     - SERVER_COLOR=#00d4ff
27     - PROCESS_DELAY=0.30
28   volumes:
29     - ./app:/app
30   working_dir: /app
31   command: sh -c "pip install --no-cache-dir flask && python app.py"
32   networks:
33     - i7-leastconn-network
34
35 i7-web-2:
36   image: python:3.9-slim
37   container_name: i7-leastconn-web-2
38   environment:
39     - SERVER_NAME=WEB SERVER 2 (i7-web-2)
40     - SERVER_GROUP=I7
41     - SERVER_COLOR=#00ff99
42     - PROCESS_DELAY=0.20
43   volumes:
44     - ./app:/app
45   working_dir: /app
46   command: sh -c "pip install --no-cache-dir flask && python app.py"
47   networks:
48     - i7-leastconn-network
49
50 i7-web-3:
51   image: python:3.9-slim
52   container_name: i7-leastconn-web-3
53   environment:
54     - SERVER_NAME=WEB SERVER 3 (i7-web-3)
55     - SERVER_GROUP=I7
56     - SERVER_COLOR=#ffd166
57     - PROCESS_DELAY=0.10
58   volumes:
59     # ... konfigurasi web-3 sampai web-7 memakai pola yang sama ...

```

Gambar 3.2 Source Code docker-compose untuk Skenario i7

```
1 events {
2     worker_connections 1024;
3 }
4
5 http {
6     upstream backend_servers {
7         # Round Robin aktif secara default.
8         server i4-web-1:5000;
9         server i4-web-2:5000;
10        server i4-web-3:5000;
11        server i4-web-4:5000;
12    }
13
14    server {
15        listen 80;
16
17        location / {
18            proxy_pass http://backend_servers;
19            proxy_set_header Host $host;
20            proxy_set_header X-Real-IP $remote_addr;
21        }
22    }
23 }
```

Gambar 3.3 Source Code Konfigurasi Nginx Round Robin

```
1 events {
2     worker_connections 1024;
3 }
4
5 http {
6     upstream backend_servers {
7         # Least Connections aktif.
8         least_conn;
9
10        server i4-web-1:5000;
11        server i4-web-2:5000;
12        server i4-web-3:5000;
13        server i4-web-4:5000;
14    }
15
16    server {
17        listen 80;
18
19        location / {
20            proxy_pass http://backend_servers;
21            proxy_set_header Host $host;
22            proxy_set_header X-Real-IP $remote_addr;
23        }
24    }
25 }
```

Gambar 3.4 Source Code Konfigurasi Nginx Least Connections

```

1 from flask import Flask, jsonify
2 import os
3 import socket
4 import time
5 from datetime import datetime
6
7 app = Flask(__name__)
8
9 SERVER_NAME = os.environ.get("SERVER_NAME", socket.gethostname())
10 SERVER_GROUP = os.environ.get("SERVER_GROUP", "unknown")
11 SERVER_COLOR = os.environ.get("SERVER_COLOR", "#00d4ff")
12 PROCESS_DELAY = float(os.environ.get("PROCESS_DELAY", "0.1"))
13
14 @app.route("/")
15 def index():
16     time.sleep(PROCESS_DELAY)
17
18     hostname = socket.gethostname()
19     timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
20
21     return f"""
22 <!DOCTYPE html>
23 <html lang="id">
24 <head>
25     <meta charset="UTF-8">
26     <title>Load Balancing Monitor</title>
27     <style>
28         * {{
29             box-sizing: border-box;
30             font-family: Arial, Helvetica, sans-serif;
31         }}
32         body {{
33             margin: 0;
34             min-height: 100vh;
35             display: flex;
36             align-items: center;
37             justify-content: center;
38             background: radial-gradient(circle at top, #1f3b57, #07111f);
39             color: #ffffff;
40         }}
41         .card {{
42             width: 440px;
43             padding: 28px;
44             border-radius: 18px;
45     # ... kode HTML monitor dilanjutkan pada file app.py ...

```

Gambar 3.5 Source Code Aplikasi Backend Flask app.py

```

1 import argparse
2 import csv
3 import re
4 import time
5 from collections import Counter
6 from concurrent.futures import ThreadPoolExecutor, as_completed
7 from pathlib import Path
8
9 import requests
10
11 DEFAULT_URL = "http://localhost:8085"
12
13 def extract_server_name(html: str) -> str:
14     pattern = r"WEB SERVER\s+\d+\s+\[^\]]+\)"
15     match = re.search(pattern, html)
16     if match:
17         return match.group(0)
18     return "Server tidak terdeteksi"
19
20 def send_request(index: int, url: str, timeout: int = 15):
21     start = time.perf_counter()
22     response = requests.get(url, timeout=timeout)
23     elapsed = time.perf_counter() - start
24     response.raise_for_status()
25     server_name = extract_server_name(response.text)
26     return index, server_name, elapsed
27
28 def main():
29     parser = argparse.ArgumentParser(description="Pengujian simulasi load balancing.")
30     parser.add_argument("--url", default=DEFAULT_URL, help="URL load balancer. Default: http://localhost:8085")
31     parser.add_argument("--requests", type=int, default=20, help="Jumlah request yang dikirim. Default: 20")
32     parser.add_argument("--workers", type=int, default=10, help="Jumlah request paralel. Default: 10")
33     parser.add_argument("--mode", default="Pengujian", help="Nama skenario, contoh: i4 Round Robin")
34     parser.add_argument("--csv", default="", help="Opsional: simpan hasil ke file CSV.")
35     args = parser.parse_args()
36
37     print(f"\n=== MULAI {args.mode.upper()} ===")
38     print(f"Target URL      : {args.url}")
39     print(f"Jumlah request   : {args.requests}")
40     print(f"Jumlah worker    : {args.workers}\n")
41
42     results = []
43
44     with ThreadPoolExecutor(max_workers=args.workers) as executor:
45         futures = [
46             executor.submit(send_request, i, args.url)
47             for i in range(1, args.requests + 1)
48         ]
49
50         for future in as_completed(futures):
51             try:
52                 index, server_name, elapsed = future.result()
53                 results.append((index, server_name, elapsed, "OK"))
54             except Exception as error:
55                 results.append((-1, str(error), 0, "ERROR"))
56
57     results.sort(key=lambda x: x[0])
58
59     # ... bagian penyimpanan CSV dan ringkasan hasil dilanjutkan ...

```

Gambar 3.6 Source Code Pengujian Request Paralel test.py

D. Metode Pengujian

Pengujian dilakukan dengan menjalankan setiap skenario secara terpisah. Skenario pertama adalah i4 Round Robin, skenario kedua adalah i4 Least Connections, skenario ketiga adalah i7 Round Robin, dan skenario keempat adalah i7 Least Connections.

Untuk menghasilkan screenshot terminal yang rapi, pengujian Round Robin sebaiknya dilakukan dengan workers bernilai 1 agar urutan server terlihat berputar secara jelas. Sementara itu, pengujian Least Connections sebaiknya dilakukan dengan workers lebih dari 1 agar terlihat pola distribusi request yang dinamis berdasarkan jumlah koneksi aktif.

Skenario	Perintah Menjalankan Container	Perintah Pengujian
----------	--------------------------------	--------------------

i4 Round Robin	<code>docker compose -f docker-compose.i4.roundrobin.yml up -d --build</code>	<code>python test.py --mode "i4 Round Robin" --requests 16 --workers 1</code>
i4 Least Connections	<code>docker compose -f docker-compose.i4.leastconn.yml up -d --build</code>	<code>python test.py --mode "i4 Least Connections" --requests 20 --workers 10</code>
i7 Round Robin	<code>docker compose -f docker-compose.i7.roundrobin.yml up -d --build</code>	<code>python test.py --mode "i7 Round Robin" --requests 21 --workers 1</code>
i7 Least Connections	<code>docker compose -f docker-compose.i7.leastconn.yml up -d --build</code>	<code>python test.py --mode "i7 Least Connections" --requests 28 --workers 14</code>

Tabel 3.1 Perintah Pengujian Setiap Skenario

BAB IV

HASIL DAN PEMBAHASAN

A. Penempatan Gambar Hasil Pengujian

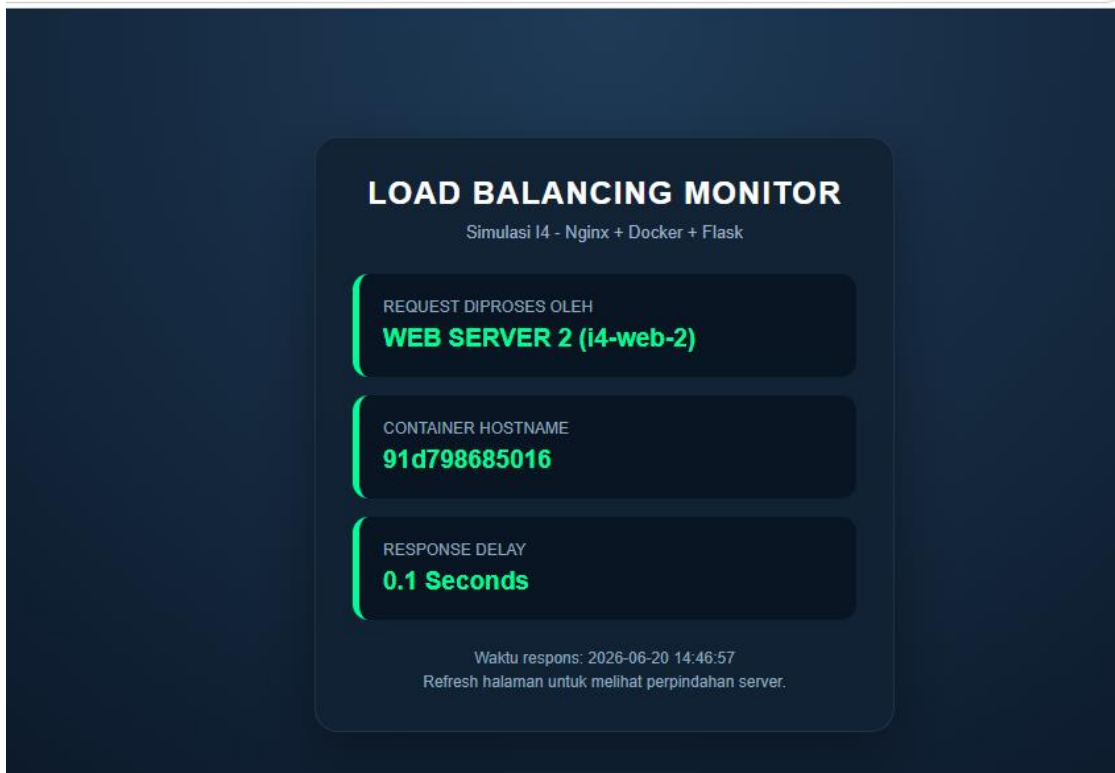
Empat gambar tampilan browser diletakkan pada bagian hasil pengujian. Urutan gambar dibuat sesuai urutan skenario agar laporan mudah dibaca dan alur pengujian terlihat jelas. Screenshot source code diletakkan pada BAB III bagian Implementasi dan Source Code, sedangkan screenshot hasil terminal diletakkan setelah gambar tampilan browser pada BAB IV.

File/Gambar	Letak di Laporan	Keterangan
gambar 1.png	Gambar 4.1	Tampilan browser skenario i4 Round Robin
gambar 2.png	Gambar 4.2	Tampilan browser skenario i4 Least Connections
gambar 3.png	Gambar 4.3	Tampilan browser skenario i7 Round Robin
gambar 4.png	Gambar 4.4	Tampilan browser skenario i7 Least Connections
SS docker-compose	Gambar 3.1 dan 3.2	Diletakkan pada BAB III bagian Implementasi
SS nginx.conf	Gambar 3.3 dan 3.4	Diletakkan sebelum pembahasan algoritma aktif
SS app.py dan test.py	Gambar 3.5 dan 3.6	Diletakkan setelah konfigurasi Nginx
SS output terminal	Gambar 4.5 sampai 4.8	Diletakkan setelah masing-masing hasil browser

Tabel 4.1 Penempatan Gambar dan Screenshot pada Laporan

B. Hasil Pengujian i4

Pada skenario i4, load balancer Nginx diarahkan ke empat backend server. Pengujian dilakukan dua kali, yaitu menggunakan Round Robin dan Least Connections. Tampilan browser digunakan sebagai bukti bahwa request berhasil diteruskan ke salah satu server backend.



Gambar 4.1 Tampilan Browser pada Skenario i4 Round Robin

Gambar 4.1 menunjukkan tampilan browser ketika skenario i4 dijalankan menggunakan algoritma Round Robin. Pada tampilan tersebut, request diproses oleh salah satu backend i4, yaitu i4-web-2. Hal ini membuktikan bahwa Nginx berhasil meneruskan request dari client ke backend server.

```

=== MULAI I4 ROUND ROBIN ===
Target URL      : http://localhost:8085
Jumlah request  : 16
Jumlah worker   : 1

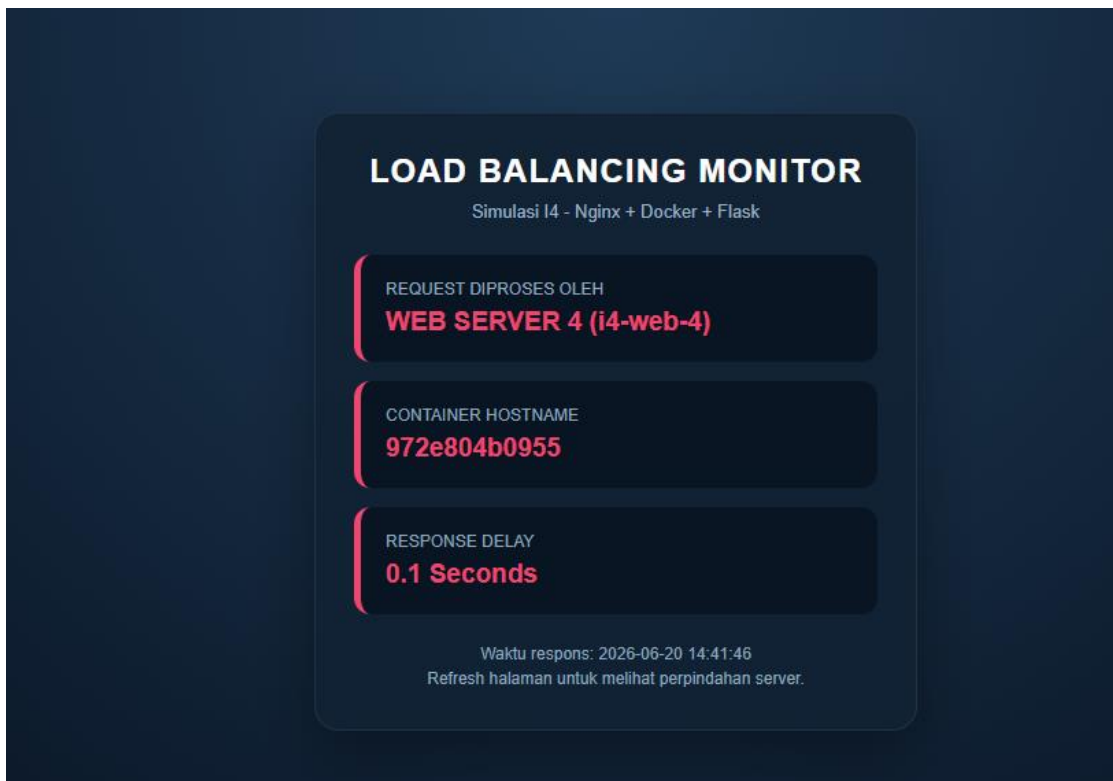
Request 01: Terarah ke -> WEB SERVER 2 (i4-web-2) | waktu: 0.111 detik
Request 02: Terarah ke -> WEB SERVER 3 (i4-web-3) | waktu: 0.207 detik
Request 03: Terarah ke -> WEB SERVER 4 (i4-web-4) | waktu: 0.105 detik
Request 04: Terarah ke -> WEB SERVER 1 (i4-web-1) | waktu: 0.304 detik
Request 05: Terarah ke -> WEB SERVER 2 (i4-web-2) | waktu: 0.105 detik
Request 06: Terarah ke -> WEB SERVER 3 (i4-web-3) | waktu: 0.205 detik
Request 07: Terarah ke -> WEB SERVER 4 (i4-web-4) | waktu: 0.104 detik
Request 08: Terarah ke -> WEB SERVER 1 (i4-web-1) | waktu: 0.304 detik
Request 09: Terarah ke -> WEB SERVER 2 (i4-web-2) | waktu: 0.104 detik
Request 10: Terarah ke -> WEB SERVER 3 (i4-web-3) | waktu: 0.209 detik
Request 11: Terarah ke -> WEB SERVER 4 (i4-web-4) | waktu: 0.108 detik
Request 12: Terarah ke -> WEB SERVER 1 (i4-web-1) | waktu: 0.304 detik
Request 13: Terarah ke -> WEB SERVER 2 (i4-web-2) | waktu: 0.104 detik
Request 14: Terarah ke -> WEB SERVER 3 (i4-web-3) | waktu: 0.204 detik
Request 15: Terarah ke -> WEB SERVER 4 (i4-web-4) | waktu: 0.104 detik
Request 16: Terarah ke -> WEB SERVER 1 (i4-web-1) | waktu: 0.304 detik

=== RINGKASAN DISTRIBUSI ===
WEB SERVER 1 (i4-web-1): 4 request
WEB SERVER 2 (i4-web-2): 4 request
WEB SERVER 3 (i4-web-3): 4 request
WEB SERVER 4 (i4-web-4): 4 request

```

Gambar 4.5 Output Terminal Pengujian i4 Round Robin

Pada Round Robin, distribusi request sebaiknya terlihat berurutan dan merata. Dengan empat backend dan 16 request, setiap server idealnya menerima empat request. Pola ini menunjukkan bahwa Round Robin membagi request berdasarkan urutan daftar server.



Gambar 4.2 Tampilan Browser pada Skenario i4 Least Connections

Gambar 4.2 menunjukkan tampilan browser ketika skenario i4 dijalankan menggunakan algoritma Least Connections. Request diproses oleh backend i4-web-4. Pada algoritma ini, pemilihan backend mempertimbangkan jumlah koneksi aktif yang sedang berjalan.

```

=== MULAI i4 LEAST CONNECTIONS ===
Target URL      : http://localhost:8085
Jumlah request  : 20
Jumlah worker   : 10

Request 01: Terarah ke -> WEB SERVER 3 (i4-web-3) | waktu: 0.213 detik
Request 02: Terarah ke -> WEB SERVER 4 (i4-web-4) | waktu: 0.107 detik
Request 03: Terarah ke -> WEB SERVER 2 (i4-web-2) | waktu: 0.109 detik
Request 04: Terarah ke -> WEB SERVER 4 (i4-web-4) | waktu: 0.116 detik
Request 05: Terarah ke -> WEB SERVER 1 (i4-web-1) | waktu: 0.311 detik
Request 06: Terarah ke -> WEB SERVER 4 (i4-web-4) | waktu: 0.108 detik
Request 07: Terarah ke -> WEB SERVER 4 (i4-web-4) | waktu: 0.120 detik
Request 08: Terarah ke -> WEB SERVER 4 (i4-web-4) | waktu: 0.116 detik
Request 09: Terarah ke -> WEB SERVER 4 (i4-web-4) | waktu: 0.110 detik
Request 10: Terarah ke -> WEB SERVER 4 (i4-web-4) | waktu: 0.112 detik
Request 11: Terarah ke -> WEB SERVER 2 (i4-web-2) | waktu: 0.109 detik
Request 12: Terarah ke -> WEB SERVER 2 (i4-web-2) | waktu: 0.109 detik
Request 13: Terarah ke -> WEB SERVER 2 (i4-web-2) | waktu: 0.108 detik
Request 14: Terarah ke -> WEB SERVER 4 (i4-web-4) | waktu: 0.118 detik
Request 15: Terarah ke -> WEB SERVER 4 (i4-web-4) | waktu: 0.109 detik
Request 16: Terarah ke -> WEB SERVER 4 (i4-web-4) | waktu: 0.113 detik
Request 17: Terarah ke -> WEB SERVER 2 (i4-web-2) | waktu: 0.110 detik
Request 18: Terarah ke -> WEB SERVER 4 (i4-web-4) | waktu: 0.111 detik
Request 19: Terarah ke -> WEB SERVER 3 (i4-web-3) | waktu: 0.207 detik
Request 20: Terarah ke -> WEB SERVER 3 (i4-web-3) | waktu: 0.206 detik

=== RINGKASAN DISTRIBUSI ===
WEB SERVER 1 (i4-web-1): 1 request
WEB SERVER 2 (i4-web-2): 5 request
WEB SERVER 3 (i4-web-3): 3 request
WEB SERVER 4 (i4-web-4): 11 request

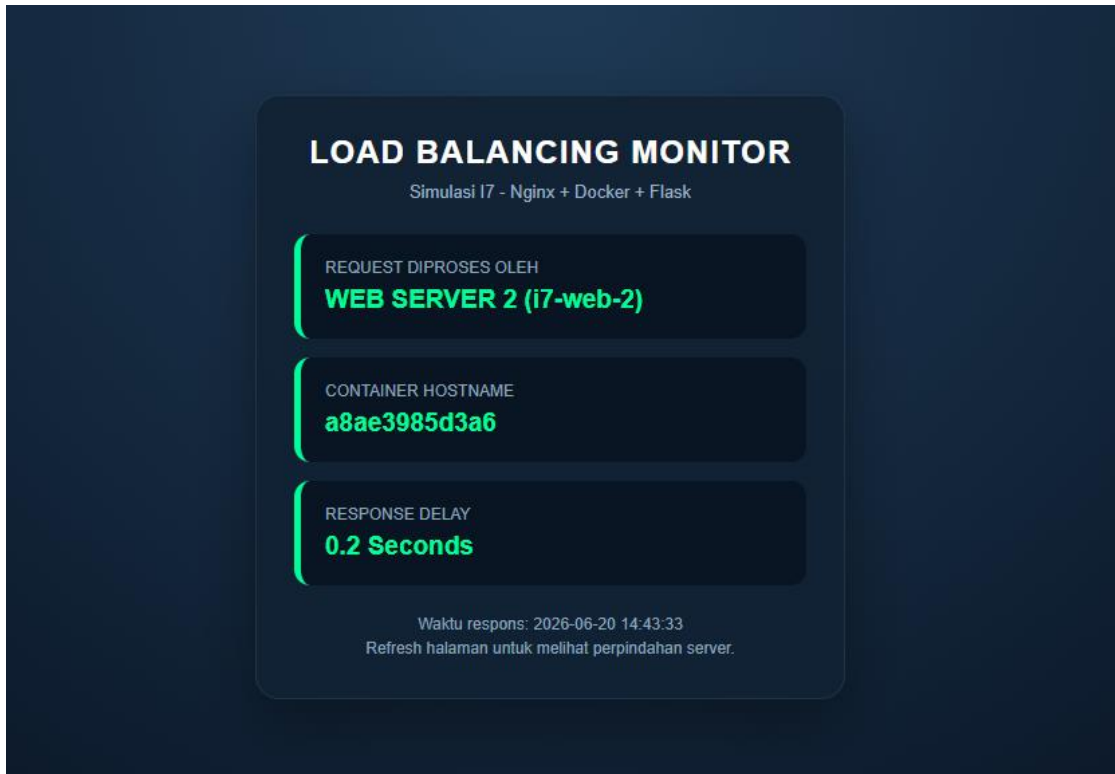
```

Gambar 4.6 Output Terminal Pengujian i4 Least Connections

Pada Least Connections, hasil terminal tidak harus berurutan. Pola yang tidak sekuensial menunjukkan bahwa Nginx memilih server berdasarkan jumlah koneksi aktif paling sedikit, bukan berdasarkan urutan daftar server semata.

C. Hasil Pengujian i7

Pada skenario i7, jumlah backend server ditingkatkan menjadi tujuh server. Tujuan skenario ini adalah melihat perilaku algoritma ketika jumlah backend lebih banyak dibandingkan i4.



Gambar 4.3 Tampilan Browser pada Skenario i7 Round Robin

Gambar 4.3 menunjukkan tampilan browser pada skenario i7 Round Robin. Request berhasil diteruskan ke backend i7-web-2. Pada pengujian Round Robin, request akan berputar dari i7-web-1 sampai i7-web-7, kemudian kembali ke server awal.

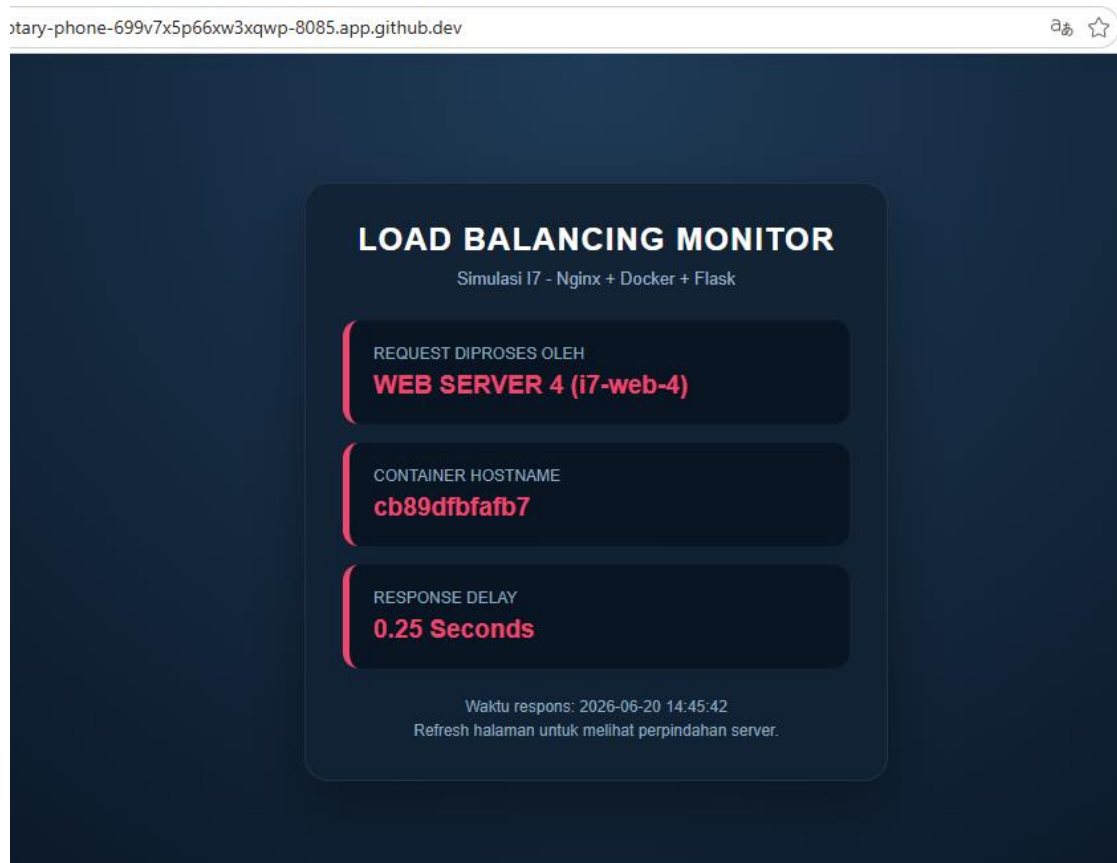
```
=== MULAI I4 ROUND ROBIN ===
Target URL      : http://localhost:8085
Jumlah request  : 16
Jumlah worker   : 1

Request 01: Terarah ke -> WEB SERVER 2 (i4-web-2) | waktu: 0.111 detik
Request 02: Terarah ke -> WEB SERVER 3 (i4-web-3) | waktu: 0.207 detik
Request 03: Terarah ke -> WEB SERVER 4 (i4-web-4) | waktu: 0.105 detik
Request 04: Terarah ke -> WEB SERVER 1 (i4-web-1) | waktu: 0.304 detik
Request 05: Terarah ke -> WEB SERVER 2 (i4-web-2) | waktu: 0.105 detik
Request 06: Terarah ke -> WEB SERVER 3 (i4-web-3) | waktu: 0.205 detik
Request 07: Terarah ke -> WEB SERVER 4 (i4-web-4) | waktu: 0.104 detik
Request 08: Terarah ke -> WEB SERVER 1 (i4-web-1) | waktu: 0.304 detik
Request 09: Terarah ke -> WEB SERVER 2 (i4-web-2) | waktu: 0.104 detik
Request 10: Terarah ke -> WEB SERVER 3 (i4-web-3) | waktu: 0.209 detik
Request 11: Terarah ke -> WEB SERVER 4 (i4-web-4) | waktu: 0.108 detik
Request 12: Terarah ke -> WEB SERVER 1 (i4-web-1) | waktu: 0.304 detik
Request 13: Terarah ke -> WEB SERVER 2 (i4-web-2) | waktu: 0.104 detik
Request 14: Terarah ke -> WEB SERVER 3 (i4-web-3) | waktu: 0.204 detik
Request 15: Terarah ke -> WEB SERVER 4 (i4-web-4) | waktu: 0.104 detik
Request 16: Terarah ke -> WEB SERVER 1 (i4-web-1) | waktu: 0.304 detik

=== RINGKASAN DISTRIBUSI ===
WEB SERVER 1 (i4-web-1): 4 request
WEB SERVER 2 (i4-web-2): 4 request
WEB SERVER 3 (i4-web-3): 4 request
WEB SERVER 4 (i4-web-4): 4 request
```

Gambar 4.7 Output Terminal Pengujian i7 Round Robin

Dengan tujuh backend dan 21 request, setiap backend idealnya menerima tiga request. Hal ini menunjukkan bahwa Round Robin mampu membagi request secara merata ketika jumlah server backend bertambah.



Gambar 4.4 Tampilan Browser pada Skenario i7 Least Connections

Gambar 4.4 menunjukkan tampilan browser pada skenario i7 Least Connections. Request diproses oleh backend i7-web-4. Pada pengujian ini, Nginx menentukan backend berdasarkan jumlah koneksi aktif paling sedikit, sehingga distribusi request dapat terlihat lebih dinamis.

```

=== MULAI I7 LEAST CONNECTIONS ===
Target URL      : http://localhost:8085
Jumlah request  : 28
Jumlah worker   : 14

Request 01: Terarah ke -> WEB SERVER 6 (i7-web-6) | waktu: 0.372 detik
Request 02: Terarah ke -> WEB SERVER 4 (i7-web-4) | waktu: 0.269 detik
Request 03: Terarah ke -> WEB SERVER 6 (i7-web-6) | waktu: 0.378 detik
Request 04: Terarah ke -> WEB SERVER 5 (i7-web-5) | waktu: 0.177 detik
Request 05: Terarah ke -> WEB SERVER 7 (i7-web-7) | waktu: 0.135 detik
Request 06: Terarah ke -> WEB SERVER 3 (i7-web-3) | waktu: 0.124 detik
Request 07: Terarah ke -> WEB SERVER 2 (i7-web-2) | waktu: 0.206 detik
Request 08: Terarah ke -> WEB SERVER 7 (i7-web-7) | waktu: 0.126 detik
Request 09: Terarah ke -> WEB SERVER 1 (i7-web-1) | waktu: 0.315 detik
Request 10: Terarah ke -> WEB SERVER 5 (i7-web-5) | waktu: 0.164 detik
Request 11: Terarah ke -> WEB SERVER 1 (i7-web-1) | waktu: 0.306 detik
Request 12: Terarah ke -> WEB SERVER 4 (i7-web-4) | waktu: 0.260 detik
Request 13: Terarah ke -> WEB SERVER 2 (i7-web-2) | waktu: 0.208 detik
Request 14: Terarah ke -> WEB SERVER 3 (i7-web-3) | waktu: 0.118 detik
Request 15: Terarah ke -> WEB SERVER 3 (i7-web-3) | waktu: 0.120 detik
Request 16: Terarah ke -> WEB SERVER 7 (i7-web-7) | waktu: 0.108 detik
Request 17: Terarah ke -> WEB SERVER 3 (i7-web-3) | waktu: 0.106 detik
Request 18: Terarah ke -> WEB SERVER 7 (i7-web-7) | waktu: 0.106 detik
Request 19: Terarah ke -> WEB SERVER 5 (i7-web-5) | waktu: 0.165 detik
Request 20: Terarah ke -> WEB SERVER 5 (i7-web-5) | waktu: 0.165 detik
Request 21: Terarah ke -> WEB SERVER 2 (i7-web-2) | waktu: 0.210 detik
Request 22: Terarah ke -> WEB SERVER 2 (i7-web-2) | waktu: 0.204 detik
Request 23: Terarah ke -> WEB SERVER 7 (i7-web-7) | waktu: 0.108 detik
Request 24: Terarah ke -> WEB SERVER 3 (i7-web-3) | waktu: 0.110 detik
Request 25: Terarah ke -> WEB SERVER 3 (i7-web-3) | waktu: 0.109 detik
Request 26: Terarah ke -> WEB SERVER 7 (i7-web-7) | waktu: 0.106 detik
Request 27: Terarah ke -> WEB SERVER 4 (i7-web-4) | waktu: 0.256 detik
Request 28: Terarah ke -> WEB SERVER 4 (i7-web-4) | waktu: 0.255 detik

```

Gambar 4.8 Output Terminal Pengujian i7 Least Connections

Pada i7 Least Connections, distribusi request dapat berbeda-beda karena setiap backend memiliki response delay yang tidak sama. Server dengan delay lebih tinggi akan menahan koneksi lebih lama, sehingga load balancer cenderung memilih server lain yang memiliki koneksi aktif lebih sedikit.

D. Analisis Perbandingan

Berdasarkan hasil pengujian, algoritma Round Robin menghasilkan distribusi request yang lebih mudah diprediksi karena bekerja berdasarkan urutan server. Pada i4, request berputar pada empat backend, sedangkan pada i7 request berputar pada tujuh backend. Algoritma ini cocok untuk kondisi server yang homogen dan beban kerja yang seragam.

Sebaliknya, algoritma Least Connections menghasilkan distribusi yang lebih dinamis. Algoritma ini tidak hanya mengikuti urutan server, tetapi memperhatikan jumlah koneksi aktif pada setiap backend. Ketika request dikirim secara paralel, backend yang sedang memiliki koneksi lebih sedikit akan lebih diprioritaskan.

Skenario	Jumlah Backend	Algoritma	Karakteristik Hasil
i4 Round Robin	4	Round Robin	Distribusi bergilir dan merata pada empat server.
i4 Least Connections	4	Least Connections	Distribusi dinamis berdasarkan koneksi aktif.

i7 Round Robin	7	Round Robin	Distribusi bergilir pada tujuh server.
i7 Least Connections	7	Least Connections	Distribusi lebih adaptif pada backend yang lebih banyak.

Tabel 4.2 Analisis Perbandingan Hasil Pengujian

BAB V

KESIMPULAN

1. Simulasi load balancing berhasil dibangun menggunakan Docker, Nginx, dan Python Flask dengan dua variasi jumlah backend, yaitu i4 dan i7.
2. Pada skenario i4, sistem menggunakan empat backend server. Pada skenario i7, sistem menggunakan tujuh backend server.
3. Algoritma Round Robin membagi request secara bergiliran berdasarkan urutan daftar server backend. Algoritma ini menghasilkan distribusi yang mudah diprediksi dan cocok untuk server yang memiliki kemampuan relatif sama.
4. Algoritma Least Connections membagi request berdasarkan jumlah koneksi aktif paling sedikit. Algoritma ini lebih dinamis dan cocok untuk beban kerja yang tidak seragam atau request yang dikirim secara bersamaan.
5. Screenshot source code sebaiknya ditempatkan pada BAB III bagian Implementasi, sedangkan screenshot tampilan browser dan output terminal ditempatkan pada BAB IV bagian Hasil dan Pembahasan.